20210410 Minjun Youn
20220049 Chaewoon Ki

1. Brief description of the 3D content and rendering

The contest suggests three types of production of 3D content: reconstructing a scene from real-world image, handcraft a 3D model, or generating it using neural models. Rather than choosing one of those, we decided to try combining them all and created a project concept around that goal.

In our project, *To 3DML and Beyond!*, we imagine that entering the room for our CS479 lecture would transport us into Andy's room from *Toy Story*. Our video begins in the hallway outside the 3DML classroom, entering the classroom door, only to find ourselves not in the classroom but in Andy's room, with the Toy Story characters moving around. As the camera turns to light them up, they freeze, realizing they are being watched.

Each part of our video was made using separate techniques and technology stacks. For the 3DML classroom(real-world reconstruction), we captured it on video and reconstructed it with Nerfstudio. Andy's room(3d scene generation) was built in Blender, where simpler furniture is modeled by hand and more complex pieces are generated with TRELLIS. The full technical breakdown of each stage is given in the 2. Technical aspects section.

2. Technical aspects

2.1. Generating 3DML Classroom

3DML Classroom (reconstructed). The room was captured as a continuous video and processed using Nerfstudio's ns-process-data pipeline [2]. This command internally invokes COLMAP to perform Structure-from-Motion, extracting 312 keyframes from the video and estimating camera poses for each. The output was used to train a Nerfacto model via Nerfstudio for 30,000 iterations. Normal prediction was enabled to improve surface geometry. After training, the NeRF volume was exported to a colored point cloud in PLY format. The raw export was then post-processed manually in Blender for trimming, repositioning, and filling. The final point cloud contains 1,958,914 points, each with RGB color, alpha, and surface normal attributes.

2.2. Generating Andy's room

Room (handcrafted). The entire room was designed manually using Blender. We created a hollow 2 m x 2 m cube. The surfaces were covered with 2D image assets. The walls were covered with the Toy Story cloud wallpaper texture [4] and the floor with laminate-wood material texture from Poly Haven [5]. Adjustments for UV mapping were done in the Shader Editor of Blender using the Mapping node, so that each texture sat at the correct scale and orientation.

Furniture (generated). All furniture was generated as 3D assets using TRELLIS, with the conditioning modality chosen according to each object's complexity. For common, structurally simple pieces such as the bed, desk, drawer and chair, we used text-to-3D generation, since a short text prompt was enough to describe the intended shape. The text prompt used for generation is provided in the Jupyter notebook in submitted code (`Trellis/generate.ipynb`). For objects that were geometrically complex or required a specific target appearance, such as the globe [6], toy chest [7], slide[13] and toy

train [8], we used image-to-3D generation with a reference image so that the output matched the shape we wanted. Each generated asset was exported as a `.glb` file and imported into Blender for placement and scaling.

Characters (generated). Because pre-made 3D assets are not allowed, we generated the Toy Story characters ourselves from 2D images. Using product photos of Disney Store's Interactive Talking Action Figures—Woody [9], Jessie [10], Rex [11], and Bullseye [12]—as image-to-3D references, we generated all four characters. A practical challenge arose from how TRELLIS represents geometry: the model generates shapes on a fixed-resolution sparse voxel grid (64^3), so when a full-body figure is fed in at once, the face occupies only a small fraction of the grid and is allocated too few voxels to reconstruct cleanly, resulting in distorted facial features. This was most pronounced for Woody, whose detailed face came out smeared when the whole figure was generated at once. To work around this, we generated Woody's face separately so that it filled the voxel grid and retained its detail, then reattached the high-detail face to the body in Blender.

2.3. Combining and Rendering

The final rendering was implemented as a Three.js webpage. We loaded the reconstructed room as a PLY point cloud and combined the GLB assets in one WebGL scene. We arranged the asset positions in Blender, then repeatedly checked the result in Three.js and adjusted rotation, scale, and placement so the point cloud, room geometry, and toy models aligned correctly. We also added several planes to the scene to make it neat.

The camera path was planned by simulating the movement in Blender. We then used Python scripts to calculate camera poses and route control points, which were transferred into Three.js as smooth Catmull-Rom camera curves. The final render includes camera movement, target interpolation, and field-of-view changes for zoom effects.

To make the scene more dynamic, we added small floating motions to the toy characters so they feel alive, inspired by Toy Story. The scene also uses Three.js lighting, fog, tone mapping, and sRGB color output to improve the final visual quality.

The project logo was created with Pixelframe's Toy Story font generator [14], a browser-based tool that uses HTML Canvas to assemble human-made assets through pre-written rules rather than AI image generation. The resulting logo was used in the video's ending and in the title of this write-up.

3. Reproduction steps

3.1. Generating 3DML Classroom

Step 1: Process Video-extract frames and recover camera poses via COLMAP SfM:

```
ns-process-data video \
  --data NeRF/data/raw/real_room_7.mp4 \
  --output-dir NeRF/data/real_room_7
```

Step 2: Train Nerfacto

```
ns-train nerfacto \
  --data NeRF/data/real_room_7 \
  --experiment-name real_room_7 \
  --max-num-iterations 30000
```

Step 3: Export Point Cloud

```
ns-export pointcloud \
  --load-config NeRF/outputs/real_room_7/nerfacto/<timestamp>/config.yml \
  --output-dir NeRF/exports/real_room_7_pointcloud
```

3.2. Generating Andy's room

The room is delivered as the final assembled scene inside the submitted webpage, while all generated assets can be reproduced from the Jupyter notebook included in the submitted code (`TRELLIS/generate.ipynb`). Reproduction therefore has two parts: regenerating the assets (fully scripted) and reassembling the room in Blender (manual; the finished file is provided).

1. Asset generation (reproducible). All furniture and characters were generated locally with TRELLIS. Set up the TRELLIS environment following the official repository [1], which requires an NVIDIA GPU with at least 16 GB of memory. The input reference images are already included under `assets/example_image/`, so the notebook can be run directly. Each cell in `generate.ipynb` records the exact inputs we used:

- *Text-to-3D* (`generate.py`) for simple pieces (e.g. bed, desk, chair and drawer) with the prompt and parameters (seed, steps, CFG strength, etc.) stored in the cell.
- *Image-to-3D* (`example.py`) for the globe, toy chest, train, slide, and the characters, each taking a reference image as input.

Every cell writes out a `.glb` mesh (and a preview `.mp4`) under the output name given in the cell. Running the notebook top to bottom reproduces all assets used in the room.

2. Room assembly in Blender (manual; file provided). The room shell and the final scene layout were built by hand in Blender and cannot be regenerated by a script, so the complete Blender project is provided as part of the submission. The manual steps were: create a 2 m × 2 m cube and hollow it out; flip the normals inward; apply the wallpaper and floor textures and adjust their scale/orientation with the Mapping node in the Shader Editor; and import each generated `.glb`, then position and scale it within the room. Opening the provided project file reproduces the assembled room exactly as rendered.

3.3. Combining and Rendering

To reproduce the rendering, first open the submitted webpage folder and install the required packages with npm install. Then run the Three.js renderer using npm run dev and open the displayed URL in a browser. The webpage automatically loads the point cloud and GLB assets, then starts the camera animation after all assets are ready.

4. Citations

4.1. Models

[1] Xiang, J., Lv, Z., Xu, S., Deng, Y., Wang, R., Zhang, B., … Yang, J. (2025). Structured 3D latents for scalable and versatile 3D generation. 2025 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 21469 – 21480. IEEE. Retrieved from https://doi.org/10.1109/cvpr52734.2025.02000

[2] Tancik, M., Weber, E., Ng, E., Li, R., Yi, B., Wang, T., … Kanazawa, A. (2023). Nerfstudio: A modular framework for neural radiance field development. *Special Interest Group on Computer Graphics and Interactive Techniques Conference Conference Proceedings*, 1 – 12. New York, NY, USA: ACM. Retrieved from https://doi.org/10.1145/3588432.3591516
[3] Three.js Authors. (2026). *Three.js* (Version r184) [Computer software]. https://github.com/mrdoob/three.js

4.2. Image & Texture Assets

[4] luxojr888. *Toy Story cloud wallpaper* [Digital artwork]. DeviantArt. https://www.deviantart.com/luxojr888/art/Toy-Story-Cloud-Wallpaper-808048773
[5] Poly Haven. *Laminate floor 03* [Texture]. Poly Haven. https://polyhaven.com/a/laminate_floor_03
[6] BSHAPPLUS. (n.d.). *13" world globe for kids and adults, blue* [Product image]. Walmart. https://www.walmart.com/ip/894394465
[7] YOLOXO. (n.d.). *Collapsible storage box / toy organizer* [Product image]. Amazon. https://www.amazon.com/dp/B0BM88KCXG
[8] Green Toys. (n.d.). *Train, blue (TRNB-1054)* [Product image]. Amazon. https://www.amazon.com/dp/B00IL7IFP6
[9] Disney Store. (n.d.). *Woody interactive talking action figure* [Product image]. Amazon. https://www.amazon.com/dp/B07QTZ8238
[10] Disney Store. (n.d.). *Jessie interactive talking action figure* [Product image]. Amazon. https://www.amazon.com/dp/B07PN7FP76
[11] Disney Store. (n.d.). *Rex interactive talking action figure* [Product image]. Amazon. https://www.amazon.com/dp/B07PPB9DHN
[12] Disney Store. (n.d.). *Bullseye figure* [Product image]. Amazon. https://www.amazon.com/dp/B0CHZB2JK8
[13] Step2. (n.d.). Game Time Sports Climber with Slide [Product image]. Amazon. https://www.amazon.com/dp/B00QZ2OQ8E
[14] Pixelframe Design. (n.d.). Toy Story font generator [Web-based design tool]. Pixelframe. https://pixelframe.design/toy-story-font-generator/